408 自学讲义

操作系统

同步、互斥与信号量

第三章 · 样章

从共享状态和先后条件出发，确定等待、通知与互斥范围。
例题就近解析；自测题在本章后部给出答案。

阅读约定
A4 单栏；正文与公式可搜索；图按原理重新绘制。
真题保留年份题号，个人错题保留实际错选。

2026 年 10 月 15 日

目录

3 同步、互斥与信号量 **1**

3.1 临界区约束什么，为什么还需要同步 1
3.2 软件互斥与锁：检查和占用必须连起来 3
3.3 信号量：先说明它代表什么，再计算初值 4
3.4 从前驱图到线程代码 6
3.5 缓冲区：容量许可与互斥范围分别建立 7
3.6 多个读者怎样作为一组申请资源 12
3.7 哲学家进餐：申请多个资源时怎样断开等待环 14
3.8 管程与条件变量：等待时必须让出改变条件的机会 15
3.9 写出并发代码后，怎样检验 17
3.10 自测：先判断，再看答案 17
3.11 题目与方法索引 18

第3章　同步、互斥与信号量

进程合作时，需要回答两个不同的问题：一项操作必须等谁完成，几个进程能否同时操作同一份共享数据。前者决定先后次序，后者决定互斥范围。信号量可以表达这两种约束，但其含义、初值和放置位置都要从题目条件推出。

本章代码是描述并发关系的伪代码。`P(S)` 与 `wait(S)` 同义，`V(S)` 与 `signal(S)` 同义；条件变量的 `wait`、`signal` 会另行定义。进程编号与数组下标从 0 开始。除特别说明，信号量采用本章的记录型语义，不假定等待队列按先进先出调度。

3.1　临界区约束什么，为什么还需要同步

从一次丢失的更新看原子操作

两个进程共享整数 `x`，初值为 0，各执行一次 `x=x+1`。一条高级语言语句未必是一条不可分割的机器操作。若每次更新分为“读入本地寄存器—加 1—写回”，就可能出现下图的交错：两个进程都读到 0，最后都写入 1，结果只有 1。

次序	进程 A	进程 B	共享 x
1	读 x：rA=0		0
2		读 x：rB=0	0
3	rA=rA+1=1		0
4		rB=rB+1=1	0
5	写 x=rA		1
6		写 x=rB	1

图 3.1: 两次更新都完成，但后一次写回覆盖了前一次结果

即使只有一个 CPU，进程也可能在读与写之间被切换出去，所以单处理机不能免除竞争。多处理机上还可能真正同时执行。这里把某段操作视为**原子操作**，是说其他并发操作不能在其内部插入并观察到破坏该操作语义的中间状态；不能仅凭“这是一行代码”认定原子性。

进程在运行态时占用处理机；在就绪态时具备运行条件，等待调度；在阻塞态时等待事件，暂时不能被调度执行。因资源不足而主动阻塞，与反复执行检查指令的忙等待不同。唤醒把阻塞进程变为就绪进程，不保证它立即运行。

资源、代码与先后关系

临界资源是在同一时段只允许一个进程使用的共享资源。**临界区**是进程中访问相应临界资源的代码段。进入区负责申请进入，退出区负责释放；二者不属于被保护的临界区本身。资源是一份共享对象，相关临界区可以有多段，分布在不同进程中。

你的错题 ERR-OS-2.3-027（王道做题本 §2.3 第27题）问：5 个并发进程涉及同一变量 A，有多少个相关临界区。按该题“每个进程有一个相关访问代码段”的模型，答案为 5，原错选为 1。这里的一份变量不能代替五段代码来计数。一般程序若有多处访问，不能仅凭进程数推出代码段总数。

互斥限制重叠：某进程正在访问临界资源时，其他需要访问它的进程须等待。**同步**规定合作中的先后关系：后续操作需要某项结果，就必须等产生该结果的操作完成。同一对进程可能同时受这两种约束。

例 3-1　共享缓冲区中的两种约束

来源：王道 2027 操作系统选择题做题本 §2.3 第11题，PDF第227页；个人错题 ERR-OS-2.3-011。

进程 A 和进程 B 通过共享缓冲区协作完成数据处理，进程 A 负责产生数据并放入缓冲区，进程 B 从缓冲区读数据并输出。进程 A 和进程 B 之间的制约关系是（　）。

A. 互斥关系　　B. 同步关系　　C. 互斥和同步关系　　D. 无制约关系

解析。 B 必须在 A 放入数据后读取，这是同步；对共享缓冲区的访问还须协调，避免操作冲突，这是互斥。答案为 **C**。你原选 B，它只表达了先后约束。

本题指出两类约束，并不意味着任何实现都必须恰好使用“一个同步量加一个互斥量”。操作是否反复执行、容量是否为 1，以及同步本身是否已排除重叠，都会影响实现；例3-4会分别处理。

怎样判断互斥方案是否合格

检查一种方案时，先找违反条件的具体交错，不要只看顺序执行的结果。

检查项	要问的问题
空闲让进	临界区无人使用，是否仍被一个不参与竞争的进程挡住?
忙则等待	一个进程尚未退出，另一个能否进入同一临界区?
有限等待	请求提出后，其他进程获准进入的次数是否存在有限上界？还须满足进程持续获得执行机会、临界区操作有限等前提。
让权等待	不能进入时，是阻塞并让出处理机，还是继续占用 CPU 检查?

前三项分别检查进入机会、互斥安全和等待上界；让权等待检查等待方式。一个自旋方案可能保证互斥，却不满足让权等待。**无死锁也不自动意味着无饥饿**：系统仍有进程前进，但某个进程可能反复被越过。

3.2 软件互斥与锁：检查和占用必须连起来

三种失败方案留下了什么问题

以下双进程模型假定单次共享变量读写原子、内存顺序与伪代码一致。`flag[i]` 表示进程 P_i 想进入。现代机器上的编译器和存储重排不在这些伪代码的默认模型内，实际程序还需相应原子变量与内存序保证。

方案	进入区的核心操作	一条足以否定它的执行情况
单标志法	等待 `turn==i`，退出时将机会交给对方	`turn=0`，但 P_0 一直做剩余区（不访问该临界资源的其他代码）工作；P_1 不能进入空闲临界区。
双标志先检查	先等对方 `flag` 为假，再置自己为真	双方都在对方置真前完成检查，随后同时进入，不能保证互斥。
双标志后检查	先置自己为真，再等对方为假	双方先置真，都等待对方退出；此时没有任何一方进入，无法执行清标志。

这些模型与反例据王道《2027年操作系统考研复习指导》§2.3.2，印刷第100—101页。先检查法的问题是“检查与声明占用分离”；简单调换两句又可能造成双方互等。

Peterson 算法怎样处理双方同时申请

Peterson 算法用 `flag` 表达意愿，用 `turn` 在双方都想进入时决定让谁先走。两个 `flag` 初值均为 `false`；对 $i \in \{0,1\}$，令 $j = 1 - i$：

```
flag[i] = true;
turn = j;
while (flag[j] && turn == j) { }  // 忙等待
访问临界资源;
flag[i] = false;
```

若只有自己请求，对方的 `flag` 为假，可直接进入。若双方都请求，两次对 `turn` 的写入总有先后；最后把机会让给对方的进程等待，另一方可进入。退出时清除自己的意愿，等待者随后通过检查。

还须排除“一个已进入，另一个又进来”的情况。假设 P_i 在临界区，故 `flag[i]` 为真；后来的 P_j 声明意愿并将 `turn` 置为 `i`，就必须等待。如果双方已在进入区竞争，后一次 `turn` 写入同样使其中一方等候。由此排除同时进入。在上述内存模型、进程继续执行且临界区有限等前提下，它也能保证有限等待；它仍在循环检查，不能让权等待。

TestAndSet 为什么能够上锁

硬件提供的 TestAndSet（TS，也称 TSL）以一次原子操作返回 `lock` 的旧值，并把 `lock` 置为真。以下只是该指令的逻辑含义，不能把普通语言实现的三句代码直接当作原子指令：

```
TestAndSet(lock):
    old = lock
    lock = true
    return old

// lock 初值 false
while (TestAndSet(lock)) { }
访问临界资源;
lock = false;
```

第一个读到假值的调用者同时把锁置真；后续调用者得到真值，继续等待。关键是“取旧值并置真”不可被其他竞争者插入，而不是 `while` 语句本身有特殊能力。Swap/XCHG 则原子交换共享的 `lock` 与进程私有的 `key`。共享 `lock` 初值为假，每次申请先令本进程的 `key=true`，再反复原子交换二者，直到 `key=false`，表示取得了交换前的空闲锁；退出时令 `lock=false`。

这类锁若一直循环，就是**自旋锁**。它避免睡眠和唤醒的切换开销，适用于持锁时间很短、持锁者能够继续运行的情形。它不保证公平，不能只从指令原子性推出有限等待。阻塞锁在不可用时使线程睡眠，更适合可能较长的等待；不能把“互斥锁”一词一概等同于自旋实现。

单处理机上，自旋者正在运行时，持锁者不能同时运行；若调度还能让持锁者重新运行，锁仍可能被释放。若相关内核区又禁止了能让持锁者运行的切换，便不能指望忙等自行结束。多处理机上，其他核可继续执行持锁者。

屏蔽本 CPU 的可屏蔽中断，只能在单处理机、相关执行路径不会主动阻塞等限定下保护短小的内核临界操作；它不能阻止另一 CPU 访问共享数据，不能用作一般多处理机互斥。开关中断是特权操作，也不能交给普通用户程序任意使用。分析原语的原子性时，应说明保护范围，不能笼统写“所有原语都靠关中断”。

3.3 信号量：先说明它代表什么，再计算初值

记录型 P、V 的完整语义

记录型信号量 S 包含整数 `value` 与等待队列。初始化只做一次；以后通过 P、V 改变状态。以下“修改计数—判断—入队或唤醒”须由系统作为原子同步操作实现；进程阻塞后并不把整个等待期间变成不可调度的原子区。

```
P(S):
    S.value = S.value - 1
    if S.value < 0:
        将当前进程加入 S 的等待队列并阻塞
        // 被唤醒后，本次 P 返回；不再次减 1

V(S):
    S.value = S.value + 1
    if S.value <= 0:
        从 S 的等待队列移出一个进程，将其置为就绪
```

P 表示申请一个许可。减 1 后为 0，说明最后一个可用许可刚被取得，**此时不阻塞**；减 1 后小于 0，才说明本次申请需要等待。V 增加一个许可；加 1 后仍小于或等于 0，说明原先有人等待，需要把这个机会交给一个等待者。V 本身不会因资源不足而阻塞。

`value>0` 表示尚未分配的许可数；`value=0` 表示没有未分配许可，也没有在该量上等待的进程；`value<0` 时，$-\text{value}$ 是等待队列长度。这些解释采用上述记录型定义。整型信号量的另一

种实现是在 S<=0 时循环检查，再取得许可；其计数通常不为负，且可能忙等，不能混用两套数值规则。

设两台同类打印机对应初值 2，A、B、C、D 先后申请，再由 A、B 归还；下图为便于追踪，假定按入队顺序唤醒。

操作（初值 2）	操作后 S.value	等待队列	本次唤醒
A 执行 P	1	空	无
B 执行 P	0	空	无
C 执行 P	-1	C	无
D 执行 P	-2	C → D	无
A 执行 V	-1	D	C
B 执行 V	0	空	D

图 3.2: 资源归还会唤醒一个等待者，计数不必先变为正数

A 归还后值为 −1，C 已得到可继续执行本次 P 的机会，D 仍等待。此时不能说“值仍是负数，所以 C 还必须再做一次 P”。唤醒 C 也不意味着 C 立即运行；其资源机会与 CPU 调度是两件事。

初值来自初态，不来自“同步量”这个名称

互斥量 mutex=1 表示只有一个进入许可。若某项一次性工作尚未完成，完成通知 done=0，后续进程先等通知。若缓冲区已有 k 个可消费的产品，产品许可应从 k 开始；容量为 n、已有 k 件时，空位许可应为 $n-k$。

例 3-2　同步信号量一定从 0 开始吗

来源：王道 2027 操作系统选择题做题本 §2.3 第18题，PDF第234页；个人错题 ERR-OS-2.3-018。

用 P、V 操作实现进程同步，信号量的初值为（　）。

A. −1　　B. 0　　C. 1　　D. 由用户确定

答案为 **D**。这里的“由用户确定”指根据资源或事件的初始状态确定，不能任意设数。B 适用于尚未发生、尚无可用结果的一次通知，不能覆盖所有同步情形。你原选 B，应检查题目有没有给出“尚未发生”这个限定。

将完成条件放在正确位置

若操作 B 必须在 A 完成后开始，设置 done=0：

```
前进程：A；V(done)；
后进程：P(done)；B；
```

若后进程先到，P 使它等待；若前进程先到，V 留下一个许可，后进程后来做 P 时也能通过。通知必须放在 A **完成之后**，不能在 A 开始前发布。互斥则是各参与者在受保护代码前 P、后 V；同步的一次 P 与产生它的一次 V 通常分布在不同进程中。因此，“P、V 要配合”不等于“必须在同一进程紧挨着出现”。

信号量通常没有互斥锁的持有者限制：一个进程可 P，另一个可 V。这允许它表达合作事件，也意味着不能把信号量与具有所有权要求的 mutex API 完全混同。

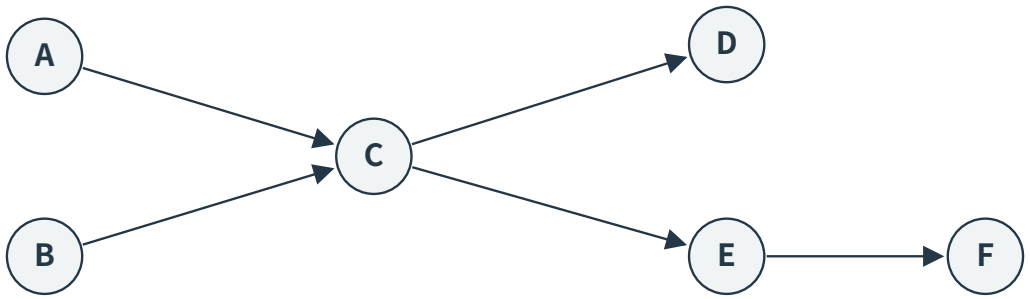
3.4 从前驱图到线程代码

前驱图的边 $A \to B$ 表示 B 不能在 A 完成前开始。先标出哪些顺序已由同一线程的程序顺序保证，再为剩下的跨线程依赖传递完成通知。给每条边各设一个初值 0 的信号量是常见的直接构造，但已有顺序可能使部分量不再需要；反过来，也不能只因两个操作都等 C，就擅自规定它们互相等待。

例 3-3　哪些边需要跨线程通知

来源：2022 年 408 第46题，8分；王道《2027年操作系统考研复习指导》§2.3 综合应用题26，印刷第126页。原题图按相同顶点和边重绘。

某进程的两个线程 T1 和 T2 并发执行 A、B、C、D、E 和 F 共 6 个操作，其中 T1 执行 A、E 和 F，T2 执行 B、C 和 D。题46图表示上述 6 个操作的执行顺序所必须满足的约束：C 在 A 和 B 完成后执行，D 和 E 在 C 完成后执行，F 在 E 完成后执行。请使用信号量的 wait()、signal() 操作描述 T1 和 T2 之间的同步关系，并说明所用信号量的作用及其初值。



T1：A、E、F 顺序执行；T2：B、C、D 顺序执行

图 3.3: 原题前驱图；图下文字标明题干给出的线程归属

解析。 在所写程序中，让 T1 按 A、E、F，T2 按 B、C、D 顺序执行，B→C、C→D、E→F 就已满足。跨线程仍需保证 A→C 和 C→E，分别设 `S_AC=0`、`S_CE=0`。

```
semaphore S_AC = 0, S_CE = 0;

T1() {                  T2() {
    A;                      B;
    signal(S_AC);           wait(S_AC);
    wait(S_CE);             C;
    E;                      signal(S_CE);
    F;                      D;
}                       }
```

T2 到达 C 前已完成本线程的 B，又收到了 A 完成的通知，因此 C 的两个前提齐备。C 完成后先发布 `S_CE`，T1 才能开始 E，随后执行 F；T2 自行继续 D。

例如，A、B、C、E、F、D 是允许的完成次序，A、B、C、D、E、F 也允许。D 与 E、F 没有额外次序要求。若把通知放到 D 后，虽然仍不违反图中的先后约束，却会强加 E 等待 D 的限制，减少题目允许的并发。不能把“运行结果安全”误当成“完整保留原约束”。

3.5 缓冲区：容量许可与互斥范围分别建立

例 3-4 同一个缓冲区为什么有不同的信号量方案

来源：2024 年 408 第46题，8分；王道《2027年操作系统考研复习指导》§2.3 综合应用题28，印刷第126页。

计算机系统中的进程之间往往需要相互协作以完成一个任务。在某网络系统中，缓冲区 B 用于存放一个数据分组，对 B 的操作有 C1、C2 和 C3。C1 将一个数据分组写入 B 中，C2 从 B 中读出一个数据分组，C3 对 B 中的数据分组进行修改。要求 B 为空时才能执行 C1，B 非空时才能执行 C2 和 C3。

（1）假设进程 P1 和 P2 均需要执行 C1，实现 C1 的代码是否为临界区？为什么？（2分）

（2）假设 B 初始为空，进程 P1 执行 C1 一次，进程 P2 执行 C2 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3分）

（3）假设 B 初始不为空，进程 P1 和 P2 各执行 C3 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3分）

解析。（1）是。两个 C1 都操作同一缓冲区，若同时发现 B 为空后分别写入，可能覆盖数据；检查并改变缓冲状态的代码需要保护。

（2）先把“初始为空、各一次”转成 C1 完成→C2 开始。设置 `done=0`，表示唯一一次写入完成的通知：

```
semaphore done = 0;
P1() { C1; signal(done); }
P2() { wait(done); C2; }
```

零个同步量不能阻止 P2 先读；一个量已经使 C2 必在 C1 之后，两次操作不会重叠，故无需再加互斥量。这里是同步关系已经保证本情境下的互斥，不能据例3-1的分类机械增加变量。

（3）B 已非空，且这里只有两个 C3，修改不会将 B 清空。无需等待新的产品，只须保证两次修改不重叠：

```
semaphore mutex = 1;
P1() { wait(mutex); C3; signal(mutex); }
P2() { wait(mutex); C3; signal(mutex); }
```

三问的共享对象相同，操作类型与执行次数却不同。“容量为 1”不能单独决定信号量个数；必须把参与者、初态和活动范围一起代入。

有限缓冲区为什么通常需要 empty、full 和 mutex

经典生产者—消费者模型中，一组生产者反复放入，一组消费者反复取走，共享容量为 $n > 0$、初始为空的缓冲区。每次放入或取出一件，操作缓冲区及其相关指针需要互斥。模型及基本算法据王道操作系统 §2.3.5，印刷第107—108页。

三项条件分别决定三种许可：`empty=n` 表示尚未被申请的空槽，`full=0` 表示尚未被申请的产品，`mutex=1` 保护单次缓冲操作。生产者在本地生产、消费者在本地使用产品，均无需占用缓冲区锁。



图 3.4: 空位许可与产品许可沿相反方向流动；四个槽只是示意，算法容量仍为 n

```
semaphore empty = n, full = 0, mutex = 1;

producer() {
    while (true) {
        x = produce();
        P(empty);          // 先取得一个空槽许可
        P(mutex);
        put(x);
        V(mutex);
        V(full);           // 放入完成，才能发布产品
    }
}
```

```
consumer() {
    while (true) {
        P(full);           // 先取得一个产品许可
        P(mutex);
        x = get();
        V(mutex);
        V(empty);
        consume(x);
    }
}
```

“尚未被申请”不能省略。容量为 2、初始为空时，一个生产者刚执行 P(empty) 而尚未放入，`empty=1`，`full=0`，物理空槽却仍有 2 个。空槽已预留，信号量先减少，实际空位数还没变；消费者取得 `full` 后、实际取出前也类似。因此不能把执行中每一步的 `empty+full` 都写成 n，也不能把 `full` 无条件等同于实存产品数。两个计数均非负，且既无已取得但尚未使用的许可，也无操作完成却尚未发布的许可时，才可把它们分别与物理空槽数、产品数对照。计数为负时，其绝对值表示相应等待者数。

为什么不能持有缓冲区锁等待产品

若消费者先 `P(mutex)` 再 `P(full)`，初始为空时，它会持锁阻塞。生产者随后申请 `mutex` 也阻塞；消费者等生产者放入，生产者等消费者解锁，形成等待环。

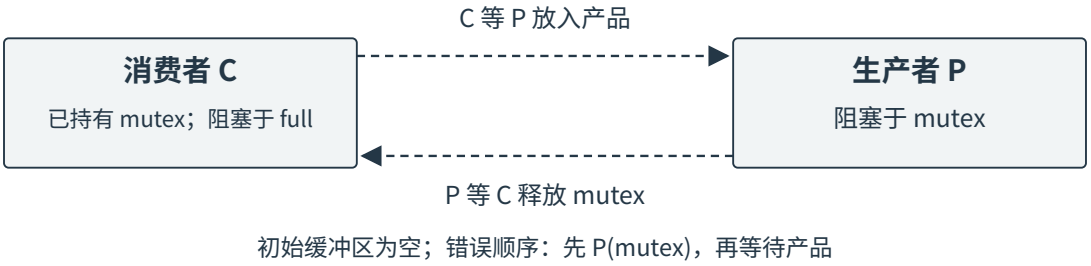


图 3.5: 错误的 P 顺序使能够改变条件的进程也被挡住

所以在这个算法中，容量或产品的 P 在外，缓冲区锁的 P 在内。判断别的算法时仍要追问：当前持有什么，等待什么，谁能使条件成立，对方是否也要用所持资源。不能把“同步 P 总在所有互斥 P 前”背成普遍定理；整批取数和读者组会给出更具体的锁范围。

放入之后，本算法的 `V(mutex)` 与 `V(full)` 可交换：消费者即使先收到产品许可，仍须取得 `mutex` 才能取出。原顺序可避免提前唤醒后再等锁。这个判断依赖放入已经完成、且后续访问也持锁；不是所有 V 都能任意挪动。

你的错题 ERR-OS-2.3-039（做题本第255页）问生产者—消费者模型解决什么问题，原选“共享资源的死锁与饥饿”，正确项是“进程之间的同步和互斥”。该模型不自动保证公平；错误的 P 顺序还可能制造死锁。避免这一错误顺序，是正确实现的条件，不是该模型能解决任意死锁的证明。

例 3-5　消费者只能取特定类别时怎样计数

来源：2009 年 408 第45题，7分；王道操作系统 §2.3 综合应用题17，印刷第123页。

三个进程 P1、P2、P3 互斥使用一个包含 N（N>0）个单元的缓冲区。P1 每次用 produce() 生成一个正整数并用 put() 送入缓冲区某一空单元中；P2 每次用 getodd() 从该缓冲区中取出一个奇数并用 countodd() 统计奇数个数；P3 每次用 geteven() 从该缓冲区中取出一个偶数并用 counteven() 统计偶数个数。请用信号量机制实现这三个进程的同步与互斥活动，并说明所定义信号量的含义（要求用伪代码描述）。

解析。“缓冲区非空”不足以允许 P2 取数：其中可能只有偶数。保留共同的空位许可，把产品许可按可取条件拆为 odd 与 even；三种缓冲操作共用 mutex。

原题未明说缓冲区初态。以下按参考答案采用**初始为空**：empty=N，odd=even=0，mutex=1。令 x 是 P1 本次生成的本地整数，put() 放入它；保留题设函数名。

```
semaphore empty = N, odd = 0, even = 0, mutex = 1;

P1() {
    while (true) {
        x = produce();
        P(empty);
        P(mutex);
        put();
        V(mutex);
        if (x % 2 == 1) V(odd); else V(even);
    }
}
```

```
P2() {
    while (true) {
        P(odd);
        P(mutex);
        getodd();
        V(mutex);
        V(empty);
        countodd();
    }
}
```

```
P3() {
    while (true) {
        P(even);
        P(mutex);
        geteven();
        V(mutex);
        V(empty);
        counteven();
    }
}
```

每次 put 只发布其所属类别的一份许可，消费者不能拿另一类别的许可代用。getodd/geteven 完成后，空位可立刻交还；各自的统计不访问共享缓冲区，留在锁外。

若初始已有 a 个奇数、b 个偶数，则应改为 odd=a，even=b，empty=N-a-b，其中 $a, b \geq 0$ 且 $a + b \leq N$。若输入长期没有某类别，该类别消费者可以持续等待；只要生产者和另一消费者仍前进，就不能仅凭这一等待认定系统死锁。题目没有保证所有类别都会出现。

例 3-6　连续取 10 件改变的是哪一层互斥

来源：2014 年 408 第47题，8分；王道操作系统 §2.3 综合应用题20，印刷第124页。

系统中有多个生产者进程和多个消费者进程，共享一个能存放 1000 件产品的环形缓冲区（初始为空）。当缓冲区未满时，生产者进程可以放入其生产的一件产品，否则等待；当缓冲区未空时，消费者进程可以从缓冲区取走一件产品，否则等待。要求一个消费者进程从缓冲区连续取出 10 件产品后，其他消费者进程才可以取产品。请使用信号量 P、V（或 wait()、signal()）操作实现进程间的互斥与同步，要求写出完整的过程，并说明所用信号量的含义和初值。

解析。“连续”限制其他消费者插入取数，不禁止生产者在这期间补货。由此需要两个范围：`mutex=1` 保护一次缓冲操作，`batch=1` 由一个消费者持有直到它取满 10 件。生产者不申请 `batch`。

```
semaphore empty = 1000, full = 0, mutex = 1, batch = 1;
item buf[1000];
int in = 0, out = 0;

producer() {
    while (true) {
        x = produce();
        P(empty);
        P(mutex);
        buf[in] = x;
        in = (in + 1) % 1000;
        V(mutex);
        V(full);
    }
}
```

```
consumer() {
    while (true) {
        P(batch);
        for (int k = 0; k < 10; ++k) {
            P(full);
            P(mutex);
            local[k] = buf[out];
            out = (out + 1) % 1000;
            V(mutex);
            V(empty);
        }
        V(batch);
        consume(local);
    }
}
```

`empty`、`full` 的含义与初值来自容量和初态。`in`/`out` 分别指向下一次放入、取出的位置；`local[0..9]` 是消费者私有的十件产品存储。每次取走一件就归还一个空槽，不必等整批结束。

假设消费者已取出 3 件，缓冲区暂时为空。它持有 `batch`，等 `full`；但它没有持有 `mutex`，生产者能继续放入并 V(full)，因此可接着取第 4 件。其他消费者不能插入，所要求的连续性得到保持。若把 `mutex` 从整批开头一直持有到末尾，消费者会在缺货时挡住补货者，形成死锁。

本解不要求开始时凑齐 10 件。不要让多个消费者在批锁之外各申请若干产品许可，再等凑够十个；这会把产品预留给多个消费者，破坏整批协调。batch 在前并非违反上一节的锁顺序：真正要检查的是生产者是否也依赖它，以及消费者等待产品时是否仍给生产者留下执行通路。

3.6 多个读者怎样作为一组申请资源

读者—写者模型允许多个读者同时读共享文件；写者之间互斥，写者与任何读者互斥。读者读完不清空文件，因此“已有数据”不应被当作一个被读走就消耗的产品许可。以下采用王道操作系统 §2.3.5 印刷第109—111页的经典模型；一次读或写有限，所有访问者遵守同一协议。

首读者申请，末读者释放

令 rw=1 表示文件访问权：一个写者独占，或者一组读者共同持有。共享计数 readCount=0 记录已登记但尚未退出的读者数，countMutex=1 保护该计数和首末读者判断。

```
semaphore rw = 1, countMutex = 1;
int readCount = 0;

reader() {
    P(countMutex);
    if (readCount == 0) P(rw);
    readCount++;
    V(countMutex);

    读文件;

    P(countMutex);
    readCount--;
    if (readCount == 0) V(rw);
    V(countMutex);
}
```

```
writer() {
    P(rw);
    写文件;
    V(rw);
}
```

第一个读者为整组取得 rw，后续读者只登记，不重复申请 rw；最后一个读者离开才归还。若每个读者都在读文件前 P(rw)、读后 V(rw)，读者也被串行化，题目允许的并发便丢失了。

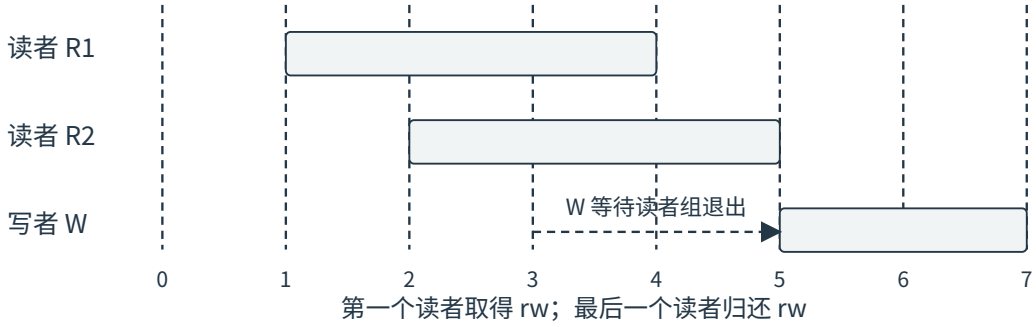


图 3.6: 重叠的两个读者共同占有一个读者组的文件访问权；横轴为示意时间

计数的检查和修改须在同一个受保护范围。假设只保护加减、不保护前面的判断：`readCount=1` 时，新读者检查到非零，准备跳过 P(rw)，此时暂停；原有的最后一个读者退出，把计数减到 0 并释放 rw；写者取得 rw 后，新读者才恢复，将计数加 1 并开始读。新读者没有申请 rw，却与写者同时访问。把判断和登记一起保护，才能排除这条交错。

首读者可能持有 `countMutex` 等待 `rw`，此处为何不必然死锁？它等待时 `readCount=0`，所以没有已登记的读者需要取得 `countMutex` 来做末读者退出；正在写的进程归还 `rw` 也不需要 `countMutex`。

读者优先与公平需要分别说明

上述算法中，只要仍有读者在读，新来的读者就能加入同一组，即使已有写者等 `rw`。不断到来的读者可能使 `readCount` 始终不归零，写者因而饥饿。

一种改法是加入口信号量 `gate=1`。每个读者在登记前 P(gate)，登记后 V(gate)；写者则 P(gate)、P(rw)，写完后依次 V(rw)、V(gate)。其余读者退出代码不变：

```
// 读者的入口，替换上一算法的入口部分
P(gate);
P(countMutex);
if (readCount == 0) P(rw);
readCount++;
V(countMutex);
V(gate);

// 写者完整过程
P(gate);
P(rw);
写文件;
V(rw);
V(gate);
```

一旦写者取得 `gate`，新读者不能再登记；原有读者仍能按原退出代码离开，最后一个释放 `rw`，写者便可进入。若 `gate` 队列按先进先出服务，且进程持续获得执行机会、读写有限，可进一步说明等待者不会被后来者无限越过。**仅有信号量而没有公平排队约定，不能无条件宣称“不会饥饿”或“严格先来先服务”**。这也不是无论到达次序如何都让写者先于读者的绝对写优先策略。

3.7 哲学家进餐：申请多个资源时怎样断开等待环

五个哲学家 P_0 至 P_4 围桌而坐，每人须同时持有左右两根筷子才能吃饭。一根筷子只供一人使用，吃完归还，再思考。统一编号：P_i 需要 C_i 与 $C_{(i+1) \bmod 5}$，每个筷子量初值为 1。经典模型据王道操作系统 §2.3.5，印刷第111—112页。

如果所有人都先 P(C[i]) 再 P(C[(i+1)%5])，一种允许的交错是每人先拿到一根。此时所有人等右邻已持有的另一根，无人能到达归还步骤。

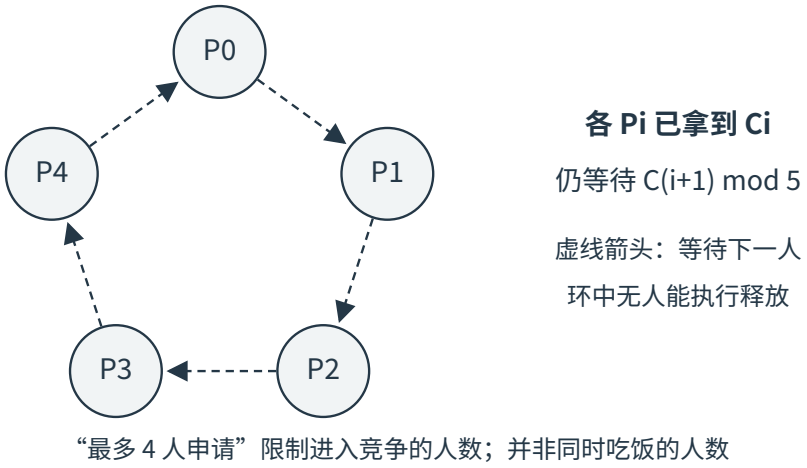


图 3.7: 每个进程只持有一根筷子，也足以形成整体死锁

限制竞争人数，与限制同时吃饭人数不同

增加 `room=4`，使最多四人处在“开始拿筷子到全部归还”的区间内：

```
semaphore room = 4;
semaphore C[5] = {1, 1, 1, 1, 1};

philosopher(i) {
    while (true) {
        思考;
        P(room);
        P(C[i]);
        P(C[(i+1) % 5]);
        吃饭;
        V(C[(i+1) % 5]);
        V(C[i]);
        V(room);
    }
}
```

若所有竞争者都各持一根并等待下一根，要形成完整环就必须五人都参与；`room=4` 阻止了这种状态。沿等待方向追下去，会遇到未占筷子的缺口或正在吃饭、能够归还筷子的人。只要持有者继续执行、进餐有限，就不会陷入上述整体等待环。

这里的 4 是同时申请区间内的人数上限。五根筷子最多支持 $\lfloor 5/2 \rfloor = 2$ 人同时吃饭，不能把“最多四人申请”误读为“四人同时进餐”。该办法排除了这个死锁模型，不自动保证某一饥饿者最终抢到许可；仍需公平条件。

另一种办法是全局排序：所有人都先取编号较小、再取较大的筷子。若存在资源等待环，沿环等待的资源编号就必须严格递增，最后又回到起点，不可能成立。每人独立“先左后右”不是全局排序；编号相邻位置在取模后更须检查。

也可用额外互斥量把“拿两根”的过程串行化，拿齐后立即释放这个额外量，吃完直接归还筷子。这能断开多人各拿一根的环，但可能让一个正在等筷子的人挡住另一个本可拿齐的人。它也不等于“先原子检查两根都空闲，然后同时取得”；若题目要求整组资源同时分配，就需要更强的原子检查与分配机制。

3.8 管程与条件变量：等待时必须让出改变条件的机会

管程把共享数据、对数据的操作过程和初始化封装在一个同步模块中，参与者通过规定入口访问共享数据；同一时刻至多一个进程在其中执行。这个互斥规则由实现管程的语言或运行系统保证，不要求调用者在每个入口外重复编写 P、V。管程不是一个自行运行的进程。

但“每次只准一人进入”还不够：消费者进入后可能发现无产品。若它在管程里循环等待并一直保有管程，生产者就无法进入改变条件。**条件变量**为这种等待提供队列；实际条件由共享数据上的布尔判断表达，例如 `count>0`。条件变量本身不是保存产品数的整数。

wait 负责等待，不替调用者判断条件

例 3-7　条件变量的 wait 做什么

来源：2018 年 408 第28题；王道 2027 操作系统选择题做题本 §2.3 第54题，PDF第271页；个人错题 ERR-OS-2.3-054。

若 x 是管程内的条件变量，则当进程执行 x.wait() 时所做的工作是（　）。

A. 实现对变量 x 的互斥访问

B. 唤醒一个在 x 上阻塞的进程

C. 根据 x 的值判断该进程是否进入阻塞态

D. 阻塞该进程，并将之插入 x 的阻塞队列中

答案为 **D**。调用者先检查共享状态，只有需要等待时才调用 x.wait()；调用后进入 x 的等待队列，并释放管程使用权，让其他进程能进入。你原选 C，该选项把“调用前的条件判断”放进了 wait 的职责，还把 x 当作可直接判值的资源计数。

管程中执行 x.signal()，是通知一个已经在 x 上等待的进程。若没有等待者，通知不积累；以后进入的进程须重新检查共享状态。信号量 V 即使没有等待者，也会使计数增加，留下一个可供后续 P 使用的许可。

你的另一道错题 ERR-OS-2.3-029（做题本 §2.3 第29题，PDF第245页）要求找错误说法。C 项“管程中 signal 操作的作用和信号量机制中的 V 操作相同”为错误，正确答案 C；原选 D。判别 C 只需检查“无等待者”的边界：前者不留下通知，后者增加许可。D 项把管程作为语法模块与可创建、撤销的进程比较，应在该教材抽象下理解；不能由其表述推出所有语言的管程对象都不能实例化。

通知以后，何时重新检查条件

不同管程定义对 signal 后的运行次序有不同规定。Hoare 式语义可立即转交管程给被唤醒者；Mesa 式语义让通知者继续，等待者以后重新竞争进入权。题目若给出特定语义，应据此推演，不能混用。

下面用 Mesa / POSIX 式条件等待说明可独立执行的做法。`wait(cv,m)` 把“释放互斥锁 m 与进入等待”衔接为原子等待操作；返回前重新取得 m。通知不保证等待者马上运行，也不保证等它重新取得 m 时条件仍成立，所以用 `while` 重检。实际接口还可能出现虚假唤醒，同样需要重检。

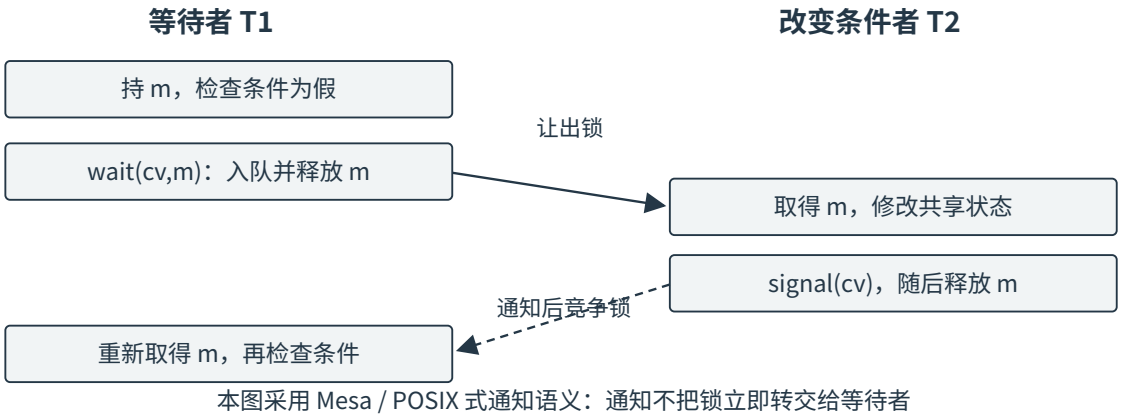


图 3.8: wait 释放锁，使另一个线程能够改变共享状态；等待返回前须重新取得锁

设这个模块的容量为 $n > 0$，初始为空，m 初始化为未加锁；插入和取出维护同一个缓冲区。

```
// 单个缓冲模块的共享状态
mutex m;
condition notEmpty, notFull;
int count = 0;

put(x) {
    lock(m);
    while (count == n) wait(notFull, m);
    插入 x; count++;
    signal(notEmpty);
    unlock(m);
}
```

```
get() {
    lock(m);
    while (count == 0) wait(notEmpty, m);
    x = 取出元素; count--;
    signal(notFull);
    unlock(m);
    return x;
}
```

若消费者发现空就调用 wait，它释放 m；生产者才能取得 m 放入，并通知消费者。被通知者回来时再次检查 `count`，是因为这期间可能已有另一个消费者先取走产品。不能把 `wait` 拆成

普通的“先 unlock，再单独入队”：另一线程可能恰在两步之间发出无人接收的通知，等待者随后入队，就错过了这次机会。

这里的接口语义用于把等待机制讲清，并非要求记忆 POSIX 参数及错误码。参见 The Open Group《Base Specifications Issue 8》条件等待与通知条目；教材管程概念、条件变量比较见王道操作系统 §2.3.6，印刷第112—114页。

3.9 写出并发代码后，怎样检验

题干中的名词不会直接决定信号量个数。先把每个进程的一次活动写清，标出它开始前必须成立的条件，再说明谁能使该条件成立。把“可用槽位”“某类产品”“阶段已经完成”等条件转成许可，把会冲突的共享状态操作放进相应互斥范围，才能确定变量及初值。

题干给出的条件	转成什么关系	应检查的代码位置
必须等某操作完成	完成通知，初值由该事件是否已发生决定	前操作完成后 V，后操作开始前 P。
最多容纳 n 件	容量许可，有放入与归还两个方向	先申请空位，放入后发布产品，取出后归还空位。
只能取某一类	每一类独立的可取条件	发布对应类别，不用总 full 冒充所有消费者的许可。
多个读者可以并行	读者组与写者互斥	保护首末读者判断，而非将整个读操作串行化。
连续取 k 件后才轮到别人	消费者整批排他	批锁覆盖 k 次，缓冲锁只覆盖单次；等待时允许补货。

最后手推几条会暴露错误的轨迹：后继先运行、缓冲区初始为空或已满、两个请求几乎同时到达、一个进程拿到许可但尚未实际操作、通知发出但被通知者尚未重新运行。每一步记下“持有—等待—可执行者”。检查安全性后，再分别检查死锁、饥饿与不必要的串行化；三者不能互相代替。

3.10 自测：先判断，再看答案

原创回顾设问：下列问题依据本章例题与经典模型编写，用于检查中间状态和适用边界。

1. 按记录型语义，P 前 `S.value=1`，P 后为 0，要阻塞吗？一次 V 使值从 −1 变为 0，要唤醒吗？被唤醒者是否还要为这次申请再减 1？
2. 在2022 年前驱题中，能否在 D 完成后才 signal(S_CE)？这样是否保留了图允许的所有并发？
3. 在分类缓冲题中，初态有 a 个奇数、b 个偶数，三个计数量怎样初始化？如果缓冲区只有偶数，P2 等待能否说明整个系统死锁？
4. 在成批取数题中，某消费者取了 3 件后暂时缺货，应继续持有哪个量、等待哪个量？生产者还能否运行？
5. 哲学家最多允许四人进入申请区，是否能有四人同时吃饭？
6. 条件变量 signal 在没有等待者时，是否留下一份以后可取得的许可？

自测答案

1. P 后为 0 不阻塞，表示取走最后一个许可。V 后为 0 仍须唤醒一个等待者；它已取得本次申请的机会，不再为同一次 P 减 1。
2. 能保证 E 不早于 C，却额外让 E 等 D，缩小并发范围。通知应在 C 完成后、D 前发出。
3. `empty=N-a-b`，`odd=a`，`even=b`；需 $a,b \geq 0$、$a+b \leq N$。只有 P2 无奇数可取，不能推出整个系统无法前进。
4. 持有 `batch`、等待 `full`，此时没有持有 `mutex`。生产者不需要 `batch`，仍能取得空位与缓冲区锁，补入产品。
5. 最多两人同时吃饭，四是申请区间内的人数上限。
6. 条件变量通知不积累为以后可消费的许可；有资源初态要用共享状态表达。

3.11 题目与方法索引

查找对象	位置
临界资源、临界区、同步、互斥；错题27并入辨析	3.1（第 1 页）
例3-1：王道 §2.3 第11题，错选 B	共享缓冲区的两种约束（第 2 页）
软件互斥反例、Peterson、TS、屏蔽中断	3.2（第 3 页）
例3-2：王道 §2.3 第18题，错选 B	初值由初态确定（第 5 页）
记录型数值、唤醒后不重复 P	3.3（第 4 页）
例3-3：2022 年408第46题	前驱图转为线程代码（第 6 页）
例3-4：2024 年408第46题	最少信号量与一次操作（第 7 页）
有限缓冲、许可预留、P 顺序；错题39并入辨析	3.5（第 7 页）
例3-5：2009 年408第45题	分类产品许可（第 9 页）
例3-6：2014 年408第47题	整批互斥与单次互斥（第 11 页）
首末读者、饥饿与入口公平	3.6（第 12 页）
资源全局排序、最多四人申请	3.7（第 14 页）
例3-7：2018 年408第28题，错选 C；错题29并入辨析	条件变量等待与通知（第 15 页）
条件→模型→方法，交错检查	3.9（第 17 页）

参考书名简写“王道操作系统”均指《2027年操作系统考研复习指导》。条件变量接口参见 The Open Group 条件等待条目和条件通知条目；缓冲模块的条件等待模型另参见 Oracle《Using Condition Variables》例18—20。